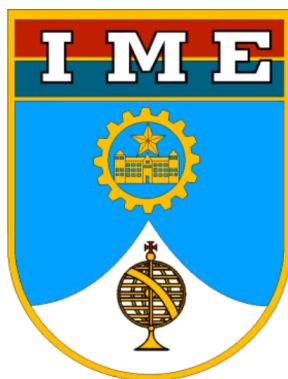


**MINISTÉRIO DA DEFESA
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
Seção de Engenharia de Defesa (SE/10)**



EE 600200: Álgebra Linear Computacional

Lista de Exercícios #1

Hermann Jacques Hernani de Oliveira
ED 26211
hermann.oliveira@ime.eb.br

Manaus, AM
Setembro de 2026

Problema #1

Questão 1.a

(a) As seguintes matrizes admitem inversa? Caso positivo, calcule a inversa.

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}.$$

Inicialmente, verifica-se a existência da inversa por meio do determinante de A :

$$\det(A) = \begin{vmatrix} 1 & 2 \\ 2 & 1 \end{vmatrix} = (1)(1) - (2)(2) = -3.$$

Como

$$\det(A) = -3 \neq 0,$$

conclui-se que A é inversível.

Para determinar A^{-1} , aplica-se o método de Gauss-Jordan à matriz aumentada $[A \mid I_2]$:

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{array} \right].$$

Aplicando a operação elementar

$$L_2 \leftarrow L_2 - 2L_1,$$

obtém-se

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -3 & -2 & 1 \end{array} \right].$$

Em seguida,

$$L_2 \leftarrow -\frac{1}{3}L_2,$$

resultando em

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 1 & \frac{2}{3} & -\frac{1}{3} \end{array} \right].$$

Finalmente, aplicando

$$L_1 \leftarrow L_1 - 2L_2,$$

tem-se

$$\left[\begin{array}{cc|cc} 1 & 0 & -\frac{1}{3} & \frac{2}{3} \\ 0 & 1 & \frac{2}{3} & -\frac{1}{3} \end{array} \right].$$

Como o lado esquerdo da matriz aumentada foi reduzido a I_2 , o lado direito corresponde a A^{-1} . Portanto,

$$A^{-1} = \begin{bmatrix} -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} \end{bmatrix}.$$

Para verificar o resultado,

$$AA^{-1} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2.$$

Logo, a matriz obtida é, de fato, a inversa de A .

A implementação em Python correspondente a esta questão está disponível na seção de Soluções Computacionais da Questão 1(a).

Questão 1.b

(b) A matriz considerada é

$$A = \begin{bmatrix} 1 & 2 & 6 \\ 2 & 1 & 6 \\ 1 & 1 & 4 \end{bmatrix}.$$

Inicialmente, verifica-se a existência da inversa por meio do determinante de A :

$$\begin{aligned} \det(A) &= 1 \begin{vmatrix} 1 & 6 \\ 1 & 4 \end{vmatrix} - 2 \begin{vmatrix} 2 & 6 \\ 1 & 4 \end{vmatrix} + 6 \begin{vmatrix} 2 & 1 \\ 1 & 1 \end{vmatrix} \\ &= (4 - 6) - 2(8 - 6) + 6(2 - 1) \\ &= -2 - 4 + 6 \\ &= 0. \end{aligned}$$

Como

$$\det(A) = 0,$$

conclui-se que A é singular e, portanto, **não admite inversa**.

Esse resultado também pode ser observado diretamente pela dependência linear entre as linhas da matriz. De fato,

$$L_3 = \frac{1}{3}(L_1 + L_2),$$

pois

$$\frac{1}{3}[(1, 2, 6) + (2, 1, 6)] = (1, 1, 4).$$

Logo, as linhas de A são linearmente dependentes, o que confirma que

$$\boxed{A^{-1} \text{ não existe.}}$$

A implementação em Python correspondente a esta questão está disponível na seção de Soluções Computacionais da Questão 1(b).

Questão 1.c

(c) A matriz considerada é

$$A = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ -3 & 4 & -5 & 2 \\ 1 & 1 & 0 & -2 \end{bmatrix}.$$

Inicialmente, verifica-se a existência da inversa. Aplicando operações elementares que não alteram o determinante, juntamente com duas trocas de linhas, obtém-se

$$\begin{aligned}
A &\sim \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 10 & -2 & 2 \\ 0 & -1 & -1 & -2 \end{bmatrix} \\
&\sim \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & -1 & -1 & -2 \\ 0 & 10 & -2 & 2 \\ 0 & 0 & -1 & 1 \end{bmatrix} \\
&\sim \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & -1 & -1 & -2 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & -30 \end{bmatrix}.
\end{aligned}$$

Como foram realizadas duas trocas de linhas, seus efeitos sobre o sinal do determinante se cancelam. Portanto,

$$\det(A) = (1)(-1)(-1)(-30) = -30 \neq 0.$$

Logo, A é **invertível**.

Para calcular A^{-1} , aplica-se o método de Gauss–Jordan à matriz aumentada $[A \mid I_4]$. Após as operações elementares,

$$[A \mid I_4] \sim \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & \frac{1}{6} & 1 & -\frac{1}{6} & \frac{1}{3} \\ 0 & 1 & 0 & 0 & \frac{3}{10} & -\frac{1}{33} & \frac{1}{10} & 0 \\ 0 & 0 & 1 & 0 & \frac{7}{30} & -\frac{2}{5} & -\frac{1}{30} & -\frac{1}{3} \\ 0 & 0 & 0 & 1 & \frac{7}{30} & \frac{2}{5} & -\frac{1}{30} & -\frac{1}{3} \end{array} \right].$$

Assim,

$$A^{-1} = \left[\begin{array}{cccc} \frac{1}{6} & 1 & -\frac{1}{6} & \frac{1}{3} \\ \frac{3}{10} & -\frac{1}{33} & \frac{1}{10} & 0 \\ \frac{7}{30} & -\frac{2}{5} & -\frac{1}{30} & -\frac{1}{3} \\ \frac{7}{30} & \frac{2}{5} & -\frac{1}{30} & -\frac{1}{3} \end{array} \right].$$

Finalmente,

$$AA^{-1} = I_4,$$

o que verifica o resultado.

A implementação em Python correspondente a esta questão está disponível na seção de Soluções Computacionais da Questão 1(c).

Problema #2

Enunciado. Seja F o subespaço de $\mathbb{R}^3$ gerado pelos vetores

$$u = (1, 1, 1) \quad \text{e} \quad v = (1, -1, -1).$$

Ache números a, b, c tais que um vetor $w = (x, y, z)$ pertence a F se, e somente se,

$$ax + by + cz = 0.$$

Solução: Como $F = \text{span}\{u, v\}$, um vetor $w \in F$ pode ser escrito como

$$w = \alpha u + \beta v, \quad \alpha, \beta \in \mathbb{R}.$$

Assim,

$$\begin{aligned} w &= \alpha(1, 1, 1) + \beta(1, -1, -1) \\ &= (\alpha + \beta, \alpha - \beta, \alpha - \beta). \end{aligned}$$

Portanto,

$$x = \alpha + \beta, \quad y = \alpha - \beta, \quad z = \alpha - \beta,$$

e, conseqüentemente,

$$y - z = 0.$$

Logo,

$$w \in F \implies y - z = 0.$$

Reciprocamente, suponha que $w = (x, y, z)$ satisfaça

$$y - z = 0.$$

Então $z = y$ e

$$w = (x, y, y).$$

Tomando

$$\alpha = \frac{x + y}{2} \quad \text{e} \quad \beta = \frac{x - y}{2},$$

tem-se

$$\begin{aligned} \alpha u + \beta v &= \frac{x + y}{2}(1, 1, 1) + \frac{x - y}{2}(1, -1, -1) \\ &= (x, y, y) \\ &= w. \end{aligned}$$

Portanto,

$$y - z = 0 \implies w \in F.$$

Conclui-se que

$$w \in F \iff y - z = 0.$$

Comparando com

$$ax + by + cz = 0,$$

uma escolha possível é

$$\boxed{a = 0, \quad b = 1, \quad c = -1.}$$

Mais geralmente, qualquer

$$(a, b, c) = (0, \lambda, -\lambda), \quad \lambda \neq 0,$$

define o mesmo subespaço.

A implementação em Python correspondente a esta questão está disponível na seção de Soluções Computacionais da Questão 2.

Problema #3

Enunciado. Seja uma transformação linear $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ que realize, nesta ordem, as seguintes operações sobre um vetor $(x, y) \in \mathbb{R}^2$:

1. escale a componente x por um fator α e a componente y por um fator β ;
2. rotacione o vetor resultante em 30° no sentido anti-horário.

(a) Calculando a matriz A que representa T .
A matriz correspondente ao escalonamento é

$$S = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix}.$$

A matriz correspondente à rotação anti-horária de 30° é

$$R = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}.$$

Como o escalonamento é realizado antes da rotação,

$$T(\mathbf{x}) = R(S\mathbf{x}),$$

e, portanto,

$$A = RS.$$

Assim,

$$\begin{aligned} A &= \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \\ &= \boxed{\begin{bmatrix} \frac{\sqrt{3}}{2}\alpha & -\frac{1}{2}\beta \\ \frac{1}{2}\alpha & \frac{\sqrt{3}}{2}\beta \end{bmatrix}}. \end{aligned}$$

(b) Caso a ordem das operações seja invertida, a matriz que representa a transformação passa a ser

$$\tilde{A} = SR.$$

Logo,

$$\tilde{A} = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2}\alpha & -\frac{1}{2}\alpha \\ \frac{1}{2}\beta & \frac{\sqrt{3}}{2}\beta \end{bmatrix}.$$

Comparando as duas matrizes,

$$A - \tilde{A} = \frac{\alpha - \beta}{2} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Portanto, em geral,

$$RS \neq SR,$$

e a inversão da ordem das operações altera o resultado.

Entretanto, se $\alpha = \beta$, o escalonamento é uniforme e $S = \alpha I$. Nesse caso,

$$RS = SR,$$

e as duas ordens produzem a mesma transformação.

Assim, os resultados são diferentes para $\alpha \neq \beta$ e coincidem no caso particular $\alpha = \beta$.

A implementação em Python correspondente a esta questão está disponível na seção de Soluções Computacionais da Questão 3.

Problema #4

Enunciado. Implemente, em Python, o algoritmo de substituição regressiva, considerando como entradas uma matriz triangular superior e um vetor coluna de dimensão correspondente. A implementação deve levantar uma exceção caso seja identificado um elemento nulo na diagonal da matriz triangular superior.

Considere o sistema linear

$$U\mathbf{x} = \mathbf{b},$$

onde $U \in \mathbb{R}^{n \times n}$ é uma matriz triangular superior. A i -ésima equação do sistema pode ser escrita como

$$u_{ii}x_i + \sum_{j=i+1}^n u_{ij}x_j = b_i.$$

Isolando x_i , obtém-se

$$x_i = \frac{b_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}, \quad i = n, n-1, \dots, 1.$$

Dessa forma, o algoritmo inicia pela última equação do sistema e calcula sucessivamente $x_n, x_{n-1}, \dots, x_1$.

Para que cada etapa seja definida, é necessário que

$$u_{ii} \neq 0, \quad i = 1, \dots, n,$$

uma vez que cada componente x_i é obtida por meio de uma divisão pelo respectivo elemento diagonal u_{ii} . Portanto, caso seja identificado um elemento nulo na diagonal, a implementação interrompe o procedimento e levanta uma exceção.

Como exemplo, considere

$$U = \begin{bmatrix} 2 & -1 & 3 \\ 0 & 4 & 2 \\ 0 & 0 & 5 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -3 \\ 6 \\ -5 \end{bmatrix}.$$

Aplicando substituição regressiva,

$$x_3 = \frac{-5}{5} = -1,$$

$$x_2 = \frac{6 - 2(-1)}{4} = 2,$$

e

$$x_1 = \frac{-3 - (-1)(2) - 3(-1)}{2} = 1.$$

Logo,

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

Implementação do código em Python:

Python — Questão 4

```

1 import numpy as np
2
3
4 def substituiçao_regressiva(U, b):
5
6     U = np.asarray(U, dtype=float)
7     b = np.asarray(b, dtype=float)
8
9     if U.ndim != 2 or U.shape[0] != U.shape[1]:
10         raise ValueError(
11             "U deve ser uma matriz quadrada."
12         )
13
14     n = U.shape[0]
15
16     if b.ndim == 2 and b.shape == (n, 1):
17         b = b[:, 0]
18
19     elif b.ndim == 1 and b.shape[0] == n:
20         pass
21
22     else:
23         raise ValueError(
24             "b deve possuir dimensao correspondente a U."
25         )
26

```

```

27     if not np.allclose(U, np.triu(U)):
28         raise ValueError(
29             "U deve ser uma matriz triangular superior."
30         )
31
32     if np.any(np.isclose(np.diag(U), 0.0)):
33         raise ZeroDivisionError(
34             "Elemento nulo encontrado na diagonal de U."
35         )
36
37     x = np.zeros(n)
38
39     for i in range(n - 1, -1, -1):
40
41         soma = np.dot(
42             U[i, i + 1:],
43             x[i + 1:]
44         )
45
46         x[i] = (
47             b[i] - soma
48         ) / U[i, i]
49
50     return x.reshape(-1, 1)
51
52
53 # Teste
54
55 U = np.array([
56     [2, -1, 3],
57     [0, 4, 2],
58     [0, 0, 5]
59 ], dtype=float)
60
61 b = np.array([
62     [-3],
63     [6],
64     [-5]
65 ], dtype=float)
66
67 x = substituicao_regressiva(U, b)
68

```

```

69 print("Solucao:")
70 print(x)
71
72 # Verificacao com NumPy
73
74 x_numpy = np.linalg.solve(U, b)
75
76 print(
77     "Resultado validado:",
78     np.allclose(x, x_numpy)
79 )
80
81
82 # Teste com elemento nulo na diagonal
83
84 U_zero = np.array([
85     [2, -1, 3],
86     [0, 0, 2],
87     [0, 0, 5]
88 ], dtype=float)
89
90 try:
91     substituicao_regressiva(U_zero, b)
92
93 except ZeroDivisionError as erro:
94     print("Erro:", erro)

```

Questão 5

Enunciado. Considere um robô planar de dois elos e duas juntas, conforme a Figura 1, com

$$L_1 = 20,0 \text{ cm} \quad \text{e} \quad L_2 = 15,0 \text{ cm}.$$

(a) Implemente uma função em Python que receba os ângulos θ_1 e θ_2 e retorne a posição do efetuador final, representada pelas coordenadas $\hat{X}_U$ e $\hat{Y}_U$, expressas em centímetros com precisão de uma casa decimal.

(b) Calcule a matriz de transformação do sistema de coordenadas do referencial do efetuador final para o sistema de coordenadas $\hat{X}_U, \hat{Y}_U$, afixado na primeira junta do robô.

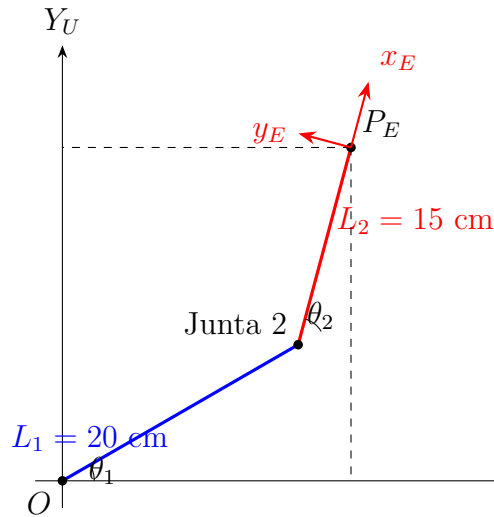


Figura 1: Configuração ilustrativa do robô planar de dois elos. Na figura, $\theta_1 = 30^\circ$ e $\theta_2 = 45^\circ$, apenas para fins de ilustração.

(a) Posição do efetuador final

A Figura 1 mostra que o primeiro elo possui orientação θ_1 em relação ao eixo X_U . Já o ângulo θ_2 é medido em relação à direção do primeiro elo; portanto, a orientação absoluta do segundo elo é

$$\theta_1 + \theta_2.$$

A posição da extremidade do primeiro elo é dada por

$$\mathbf{p}_1 = \begin{bmatrix} L_1 \cos \theta_1 \\ L_1 \sin \theta_1 \end{bmatrix}.$$

Por sua vez, o deslocamento produzido pelo segundo elo, expresso no referencial X_U, Y_U , é

$$\mathbf{p}_2 = \begin{bmatrix} L_2 \cos(\theta_1 + \theta_2) \\ L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$

Logo, a posição do efetuador final é

$$\mathbf{p}_E = \mathbf{p}_1 + \mathbf{p}_2,$$

isto é,

$$\mathbf{p}_E = \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$

Substituindo $L_1 = 20$ cm e $L_2 = 15$ cm, obtém-se

$$\begin{aligned} X_U &= 20 \cos \theta_1 + 15 \cos(\theta_1 + \theta_2), \\ Y_U &= 20 \sin \theta_1 + 15 \sin(\theta_1 + \theta_2). \end{aligned}$$

Essas expressões representam a cinemática direta do manipulador planar de dois elos.

(θ_1, θ_2)	Interpretação geométrica	(X_U, Y_U)
$(0^\circ, 0^\circ)$	braço totalmente estendido sobre X_U	$(35.0, 0.0)$
$(0^\circ, 90^\circ)$	primeiro elo horizontal e segundo vertical	$(20.0, 15.0)$
$(90^\circ, 0^\circ)$	braço totalmente estendido sobre Y_U	$(0.0, 35.0)$

Tabela 1: Casos de teste para validação da cinemática direta do manipulador.

A Tabela 1 confirma a coerência geométrica das expressões obtidas.

Na implementação computacional, os ângulos são fornecidos em radianos, conforme a convenção das funções trigonométricas do NumPy, e as coordenadas retornadas são arredondadas para uma casa decimal.

Assumindo que as duas juntas possam girar livremente, isto é, que não existam restrições adicionais aos valores de θ_1 e θ_2 , a distância do efetuador final à origem pode variar entre

$$r_{\min} = |L_1 - L_2| = 5 \text{ cm}$$

e

$$r_{\max} = L_1 + L_2 = 35 \text{ cm.}$$

Dessa forma, o espaço de trabalho do efetuador corresponde à região anular ilustrada na Figura 2.

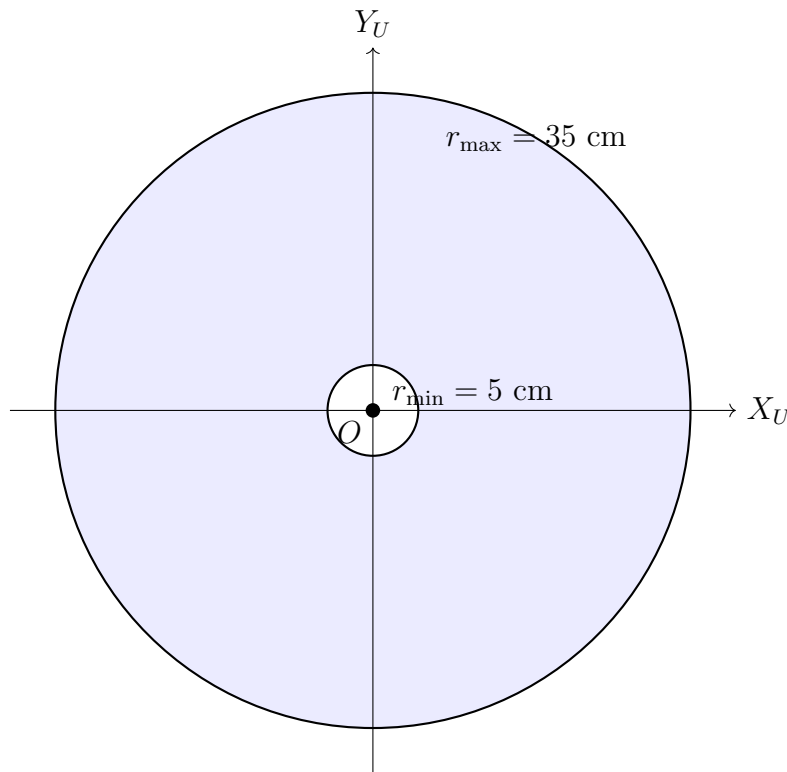


Figura 2: Espaço de trabalho do efetuador final. Como $L_1 = 20 \text{ cm}$ e $L_2 = 15 \text{ cm}$, a distância da ponta à base pode variar entre $|L_1 - L_2| = 5 \text{ cm}$ e $L_1 + L_2 = 35 \text{ cm}$.

A Figura 2 ilustra o espaço de trabalho do robô. Embora essa figura não seja exigida explicitamente pelo enunciado, ela ajuda a visualizar o conjunto de posições que o efetuador final pode alcançar.

Python — Questão 5

```

1 import numpy as np
2
3
4 def posicao_efetuador(theta1, theta2, arredondar=True):
5     """

```

```

6      Calcula a posicao do efetuador final de um robo planar
7      de dois elos.
8
9      Parametros
10     -----
11     theta1 : float
12             Angulo da primeira junta, em radianos.
13
14     theta2 : float
15             Angulo da segunda junta em relacao ao primeiro elo,
16             em radianos.
17
18     arredondar : bool
19             Se True, retorna as coordenadas com uma casa decimal.
20
21     Retorna
22     -----
23     tuple
24         Coordenadas (X_U, Y_U) do efetuador final, em cm.
25     """
26
27     L1 = 20.0
28     L2 = 15.0
29
30     # Orientacao absoluta do segundo elo
31     phi = theta1 + theta2
32
33     x_u = (
34         L1 * np.cos(theta1)
35         + L2 * np.cos(phi)
36     )
37
38     y_u = (
39         L1 * np.sin(theta1)
40         + L2 * np.sin(phi)
41     )
42
43     if arredondar:
44         return round(float(x_u), 1), round(float(y_u), 1)
45
46     return float(x_u), float(y_u)
47

```

```

48
49 def transformacao_efetuador(theta1, theta2):
50     """
51     Calcula a matriz de transformacao homogenea do
52     referencial do efetuador final E para o referencial U.
53
54     Parametros
55     -----
56     theta1 : float
57         Angulo da primeira junta, em radianos.
58
59     theta2 : float
60         Angulo da segunda junta em relacao ao primeiro elo,
61         em radianos.
62
63     Retorna
64     -----
65     numpy.ndarray
66         Matriz homogenea 3x3 que representa a pose do
67         efetuador final em relacao ao referencial U.
68     """
69
70     # Posicao sem arredondamento para preservar a precisao
71     x_u, y_u = posicao_efetuador(
72         theta1,
73         theta2,
74         arredondar=False
75     )
76
77     # Orientacao absoluta do efetuador
78     phi = theta1 + theta2
79
80     T_UE = np.array([
81         [
82             np.cos(phi),
83             -np.sin(phi),
84             x_u
85         ],
86         [
87             np.sin(phi),
88             np.cos(phi),
89             y_u

```

```

90         ],
91         [
92             0.0,
93             0.0,
94             1.0
95         ]
96     ])
97
98     return T_UE
99
100
101 # =====
102 # TESTE 1
103 # theta1 = 0 graus
104 # theta2 = 0 graus
105 # =====
106
107 theta1 = np.deg2rad(0.0)
108 theta2 = np.deg2rad(0.0)
109
110 posicao = posicao_efetuador(theta1, theta2)
111 T = transformacao_efetuador(theta1, theta2)
112
113 print("Teste 1")
114 print("Posicao do efetuador:")
115 print(posicao)
116
117 print("\nMatriz de transformacao:")
118 print(np.round(T, 10))
119
120
121 # Resultado esperado:
122 # posicao = (35.0, 0.0)
123
124 assert posicao == (35.0, 0.0)
125
126 T_esperada_1 = np.array([
127     [1.0, 0.0, 35.0],
128     [0.0, 1.0, 0.0],
129     [0.0, 0.0, 1.0]
130 ])
131

```

```

132 assert np.allclose(T, T_esperada_1)
133
134
135 # =====
136 # TESTE 2
137 # theta1 = 0 graus
138 # theta2 = 90 graus
139 # =====
140
141 theta1 = np.deg2rad(0.0)
142 theta2 = np.deg2rad(90.0)
143
144 posicao = posicao_efetuador(theta1, theta2)
145 T = transformacao_efetuador(theta1, theta2)
146
147 print("\nTeste 2")
148 print("Posicao do efetuador:")
149 print(posicao)
150
151 print("\nMatriz de transformacao:")
152 print(np.round(T, 10))
153
154
155 # Resultado esperado:
156 # posicao = (20.0, 15.0)
157
158 assert posicao == (20.0, 15.0)
159
160 T_esperada_2 = np.array([
161     [0.0, -1.0, 20.0],
162     [1.0,  0.0, 15.0],
163     [0.0,  0.0,  1.0]
164 ])
165
166 assert np.allclose(T, T_esperada_2)
167
168 print("\nTodos os testes foram concluidos com sucesso.")

```

(b) Matriz de transformação do referencial do efetuador final

Seja E o referencial do efetuador final, com origem na ponta do robô, eixo x_E alinhado com o segundo elo e eixo y_E perpendicular a x_E , no sentido anti-horário, conforme indicado na Figura 1.

A orientação absoluta do referencial E em relação ao referencial U é

$$\phi = \theta_1 + \theta_2.$$

Em coordenadas homogêneas, uma transformação plana é representada por uma matriz da forma

$$T = \begin{bmatrix} R & \mathbf{p} \\ 0 & 1 \end{bmatrix},$$

em que R é a matriz de rotação e $\mathbf{p}$ é o vetor de translação.

Para construir a transformação do referencial do efetuador final para o referencial da base, define-se inicialmente um referencial intermediário na segunda junta, alinhado com o primeiro elo. Assim,

$${}^U T_1 = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & L_1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & L_1 \sin \theta_1 \\ 0 & 0 & 1 \end{bmatrix},$$

e

$${}^1 T_E = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & L_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & L_2 \sin \theta_2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Logo,

$${}^U T_E = {}^U T_1 {}^1 T_E.$$

Efetuando a multiplicação das matrizes, obtém-se

$${}^U T_E = \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \\ 0 & 0 & 1 \end{bmatrix}.$$

Substituindo $L_1 = 20$ cm e $L_2 = 15$ cm, resulta

$${}^U T_E = \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 20 \cos \theta_1 + 15 \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 20 \sin \theta_1 + 15 \sin(\theta_1 + \theta_2) \\ 0 & 0 & 1 \end{bmatrix}.$$

Observa-se que a última coluna da matriz contém exatamente as coordenadas do efetuador final obtidas no item (a), enquanto o bloco 2×2 superior esquerdo representa a orientação do referencial E em relação ao referencial U .

Consequentemente, se um ponto é representado em coordenadas homogêneas no referencial do efetuador por

$${}^E \mathbf{p} = \begin{bmatrix} x_E \\ y_E \\ 1 \end{bmatrix},$$

suas coordenadas no referencial da base são obtidas por

$${}^U \mathbf{p} = {}^U T_E {}^E \mathbf{p}.$$

Assim, a matriz ${}^U T_E$ transforma coordenadas expressas no referencial do efetuador E em coordenadas expressas no referencial da base U .

Como exemplo, para $\theta_1 = 0^\circ$ e $\theta_2 = 90^\circ$, tem-se

$${}^U T_E = \begin{bmatrix} 0 & -1 & 20 \\ 1 & 0 & 15 \\ 0 & 0 & 1 \end{bmatrix},$$

o que indica que a origem do referencial do efetuador encontra-se na posição $(20, 15)$ e que sua orientação em relação ao sistema da base é de 90° .

As resoluções em Python apresentadas ao longo deste trabalho podem ser encontradas, na íntegra, no repositório disponível em:

Repositório GitHub — Lista 1 de Álgebra Linear Computacional.

Soluções Computacionais

Questão 1(a)

Python — Questão 1(a)

```
1 import numpy as np
2
3 A = np.array([
4     [1, 2],
5     [2, 1]
6 ], dtype=float)
7
8 det_A = np.linalg.det(A)
9
10 print(f"det(A) = {det_A:.10f}")
11
12 if np.isclose(det_A, 0.0):
13     print("A matriz A nao e inversivel.")
14 else:
15     A_inv = np.linalg.inv(A)
16
17     print("A^-1 =")
18     print(A_inv)
19
20     verificacao = A @ A_inv
21
22     print("A @ A^-1 =")
23     print(np.round(verificacao, 10))
24
25     assert np.allclose(verificacao, np.eye(2))
```

Questão 1(b)

Python — Questão 1(b)

```
1 import numpy as np
2
3 A = np.array([
4     [1, 2, 6],
5     [2, 1, 6],
```

```

6     [1, 1, 4]
7 ], dtype=float)
8
9 # Determinante
10 det_A = np.linalg.det(A)
11
12 print(f"det(A) = {det_A:.10f}")
13
14 # Posto da matriz
15 rank_A = np.linalg.matrix_rank(A)
16
17 print(f"posto(A) = {rank_A}")
18
19 # Verificacao de inversibilidade
20 if np.isclose(det_A, 0.0):
21     print("A matriz A nao e inversivel.")
22 else:
23     A_inv = np.linalg.inv(A)
24
25     print("A^-1 =")
26     print(A_inv)

```

Questão 1(b) implementação alternativa

Python — Questão 1(b) implementação alternativa

```

1 import numpy as np
2
3 A = np.array([
4     [1, 2, 6],
5     [2, 1, 6],
6     [1, 1, 4]
7 ], dtype=float)
8
9 try:
10     A_inv = np.linalg.inv(A)
11     print(A_inv)
12
13 except np.linalg.LinAlgError:
14     print("Nao foi possivel calcular a inversa: a matriz e

```

```
singular.")
```

Questão 1(c)

Python — Questão 1(c)

```
1 import numpy as np
2
3 A = np.array([
4     [ 1, 2, 1, 0],
5     [ 0, 0, -1, 1],
6     [-3, 4, -5, 2],
7     [ 1, 1, 0, -2]
8 ], dtype=float)
9
10 det_A = np.linalg.det(A)
11
12 print(f"det(A) = {det_A:.10f}")
13
14 if np.isclose(det_A, 0.0):
15     print("A matriz A nao e inversivel.")
16 else:
17     A_inv = np.linalg.inv(A)
18
19     print("\nA^-1 =")
20     print(A_inv)
21
22     verificacao = A @ A_inv
23
24     print("\nA @ A^-1 =")
25     print(np.round(verificacao, 10))
26
27     assert np.allclose(verificacao, np.eye(4))
28
29     print("\nVerificacao concluida: A @ A^-1 = I.")
```

Questão 2

Verificando uma direção

Python — Questão 2

```
1 import numpy as np
2
3 u = np.array([1.0, 1.0, 1.0])
4 v = np.array([1.0, -1.0, -1.0])
5
6 # Coeficientes arbitrarios da combinacao linear
7 alpha = 3.0
8 beta = -2.0
9
10 # Vetor pertencente a F
11 w = alpha * u + beta * v
12
13 x, y, z = w
14
15 print("w =", w)
16 print("y - z =", y - z)
17
18 assert np.isclose(y - z, 0.0)
19
20 print("O vetor pertence a F e satisfaz y - z = 0.")
```

Verificando também a direção contrária

Python — Questão 2

```
1 import numpy as np
2
3 u = np.array([1.0, 1.0, 1.0])
4 v = np.array([1.0, -1.0, -1.0])
5
6 # Vetor que satisfaz y = z
7 w = np.array([7.0, 2.0, 2.0])
8
9 x, y, z = w
10
11 if np.isclose(y - z, 0.0):
```

```

12
13     alpha = (x + y) / 2
14     beta = (x - y) / 2
15
16     w_reconstruido = alpha * u + beta * v
17
18     print("alpha =", alpha)
19     print("beta =", beta)
20
21     print("w =", w)
22     print("w reconstruido =", w_reconstruido)
23
24     assert np.allclose(w, w_reconstruido)
25
26     print("w pertence ao subespaco F.")
27
28 else:
29     print("w nao pertence a F.")

```

Questão 3

Python — Questão 3

```

1 import numpy as np
2
3
4 def matrizes_transformacao(alpha, beta):
5     """
6     Calcula as matrizes correspondentes a:
7
8     1. escala seguida de rotacao;
9     2. rotacao seguida de escala.
10    """
11
12    theta = np.deg2rad(30.0)
13
14    S = np.array([
15        [alpha, 0.0],
16        [0.0, beta]
17    ])
18

```

```

19     R = np.array([
20         [np.cos(theta), -np.sin(theta)],
21         [np.sin(theta), np.cos(theta)]
22     ])
23
24     # Primeiro escala, depois rotaciona
25     A = R @ S
26
27     # Primeiro rotaciona, depois escala
28     A_invertida = S @ R
29
30     return A, A_invertida
31
32
33 # Valores utilizados apenas para teste
34 alpha = 2.0
35 beta = 3.0
36
37 A, A_invertida = matrizes_transformacao(alpha, beta)
38
39 print("Escala seguida de rotacao:")
40 print(A)
41
42 print("\nRotacao seguida de escala:")
43 print(A_invertida)
44
45 print("\nAs matrizes sao iguais?")
46 print(np.allclose(A, A_invertida))

```

Questão 4

A implementação em Python correspondente a esta questão está disponível diretamente na solução da Questão 4.

Questão 5

A implementação em Python correspondente a esta questão está disponível diretamente na solução da Questão 5.